

tf.Tensor Stay organized with collections Save and categorize content based on your preferences.

https://www.tensorflow.org/api_docs/python/tf/Tensor

A tf.Tensor represents a multidimensional array of elements.

Code Examples

```
# Compute some values using a Tensor
c = tf.constant([[1.0, 2.0], [3.0, 4.0]])
d = tf.constant([[1.0, 1.0], [0.0, 1.0]])
e = tf.matmul(c, d)
print(e)
tf.Tensor(
[[1. 3.]
 [3. 7.]], shape=(2, 2), dtype=float32)
```

```
# Compute some values using a Tensor
```

```
c = tf.constant([[1.0, 2.0], [3.0, 4.0]])
```

```
d = tf.constant([[1.0, 1.0], [0.0, 1.0]])
```

```
e = tf.matmul(c, d)
```

Documentation

A `tf.Tensor` represents a multidimensional array of elements.

View aliases

Main aliases

`tf.experimental.numpy.ndarray`

See

Migration guide for
more details.

`tf.compat.v1.Tensor`

All elements are of a single known data type.

When writing a TensorFlow program, the main object that is manipulated and passed around is the `tf.Tensor`.

A `tf.Tensor` has the following properties:

- a single data type (`float32`, `int32`, or `string`, for example)
- a shape

TensorFlow supports eager execution and graph execution. In eager execution, operations are evaluated immediately. In graph execution, a computational graph is constructed for later

evaluation.

TensorFlow defaults to eager execution. In the example below, the matrix multiplication results are calculated immediately.

Note that during eager execution, you may discover your Tensors are actually of type `EagerTensor`. This is an internal detail, but it does give you access to a useful function, `numpy`:

In TensorFlow, `tf.functions` are a common way to define graph execution.

A Tensor's shape (that is, the rank of the Tensor and the size of each dimension) may not always be fully known. In `tf.function` definitions, the shape may only be partially known.

Most operations produce tensors of fully-known shapes if the shapes of their inputs are also fully known, but in some cases it's only possible to find the shape of a tensor at execution time.

A number of specialized tensors are available: see `tf.Variable`, `tf.constant`, `tf.placeholder`, `tf.sparse.SparseTensor`, and `tf.RaggedTensor`.

For more on Tensors, see the guide.

Attributes

`tf.Tensor.shape` is equivalent to `tf.Tensor.get_shape()`.

In a `tf.function` or when building a model using

`tf.keras.Input`, they return the build-time shape of the tensor, which may be partially unknown.

A `tf.TensorShape` is not a tensor. Use `tf.shape(t)` to get a tensor containing the shape, calculated at runtime.

See `tf.Tensor.get_shape()`, and `tf.TensorShape` for details and examples.

Methods

[View source](#)

Evaluates this tensor in a Session.

Calling this method will execute all preceding operations that produce the inputs needed for the operation that produces this tensor.

experimental_ref

[View source](#)

DEPRECATED FUNCTION

get_shape

[View source](#)

Returns a `tf.TensorShape` that represents the shape of this tensor.

In eager execution the shape is always fully-known.

`tf.Tensor.get_shape()` is equivalent to `tf.Tensor.shape`.

When executing in a `tf.function` or building a model using `tf.keras.Input`, `Tensor.shape` may return a partial shape (including `None` for unknown dimensions). See `tf.TensorShape` for more details.

The shape is computed using shape inference functions that are registered for each `tf.Operation`.

The returned `tf.TensorShape` is determined at build time, without executing the underlying kernel. It is not a `tf.Tensor`. If you need a shape tensor, either convert the `tf.TensorShape` to a `tf.constant`, or use the `tf.shape(tensor)` function, which returns the tensor's shape at execution time.

This is useful for debugging and providing early errors. For example, when tracing a `tf.function`, no ops are being executed, shapes may be unknown (See the Concrete Functions Guide for details).

The shape inference functions propagate shapes to the extent possible:

Tracing may fail if a shape mismatch can be detected:

In some cases, the inferred shape may have unknown dimensions. If the caller has additional information about the values of these dimensions, `tf.ensure_shape` or `Tensor.set_shape()` can be used to augment the inferred shape.

[View source](#)

Returns a hashable reference object to this Tensor.

The primary use case for this API is to put tensors in a set/dictionary. We can't put tensors in a set/dictionary as `tensor.__hash__()` is no longer available starting Tensorflow 2.0.

The following will raise an exception starting 2.0

Instead, we can use `tensor.ref()`.

Also, the reference object provides `.deref()` function that returns the original Tensor.

set_shape

[View source](#)

Updates the shape of this tensor.

With eager execution this operates as a shape assertion.

Here the shapes match:

Passing a `None` in the new shape allows any value for that axis:

An error is raised if an incompatible shape is passed.

When executing in a `tf.function`, or building a model using `tf.keras.Input`, `Tensor.set_shape` will merge the given shape with the current shape of this tensor, and set the tensor's shape to the merged value (see `tf.TensorShape.merge_with` for details):

Dimensions set to None are not updated:

The main use case for this is to provide additional shape information that cannot be inferred from the graph alone.

For example if you know all the images in a dataset have shape [28,28,3] you can set it with `tf.set_shape`:

Trace the function, see the Concrete Functions Guide for details.

Similarly the `tf.io.parse_tensor` function could return a tensor with any shape, even the `tf.rank` is unknown. If you know that all your serialized tensors will be 2d, set it with `set_shape`:

Trace the function

Make sure it works:

`## __abs__`

[View source](#)

`__add__`

[View source](#)

`__and__`

[View source](#)

`__array__`

[View source](#)

__bool__

[View source](#)

Dummy method to prevent a tensor from being used as a Python bool.

This overload raises a `TypeError` when the user inadvertently treats a `Tensor` as a boolean (most commonly in an `if` or `while` statement), in code that was not converted by `AutoGraph`. For example:

__div__

[View source](#)

__eq__

[View source](#)

__floordiv__

[View source](#)

__ge__

Returns the truth value of $(x \geq y)$ element-wise.

Example:

```
## __getitem__
```

[View source](#)

Overload for `Tensor.getitem`.

This operation extracts the specified region from the tensor.

The notation is similar to NumPy with the restriction that currently only support basic indexing. That means that using a non-scalar tensor as input is not currently allowed.

Some useful examples:

- `tf.newaxis` is `None` as in NumPy.
- An implicit